

- **Project Title:** Automated Plant Monitoring and Alert System
 - **Team Members (Name, Admin No.):** Ong Li Heng, 252963s, and Goh Ziqian, 243339f
 - **Module Group:** PB
 - **Tutor:** Dr Ng Kok Poh
 - **Submission Date:** 15 February 2026
-

Table of Contents

Page No.

1. Executive Summary -----	3
2. Introduction -----	4
2.1. Background-----	4
2.2. Problem statement-----	4
3. Project Objectives-----	5
4. Proposed Solution-----	6, 7, 8
4.1. Solution Overview-----	6, 7
4.2. System Architecture-----	7,8
5. Methodology-----	8, 9
5.1. Hardware Setup-----	8, 9
5.2. Software Setup-----	9
6. Design and Implementation-----	10-13
6.1. System Design -----	10
6.2. Sensor Integration-----	11
6.3. Data Processing -----	12
6.4. Database Implementation-----	13
6.5. Web Interface-----	13
7. Testing and Results-----	14-15
7.1. Test Cases-----	14
7.2. Challenges Encountered and Resolution-----	15
8. Conclusion-----	16
9. Reference-----	17
10. Appendix -----	18-22

1. Executive Summary

The Automated Plant Monitoring and Alert System is an IoT-based project to support indoor plant cultivation by monitoring surrounding environmental conditions and providing timely alerts. Due to limited land, Singapore needs to rely on indoor farming, like vertical farming, to enhance its food security by reducing its reliance on food imports. However, indoor plant cultivation requires farmers to constantly monitor plant conditions to ensure that the conditions are optimal for plant growth.

This project uses multiple sensors, such as a temperature sensor, light sensor, and ultrasonic sensor, to continuously monitor the conditions of the plant's environment. A rotary angle sensor is used to adjust the LED light brightness based on user preference. The Arduino collects sensor data and sends it to the Raspberry Pi for processing and storing the data in a MariaDB database. This system provides real-time feedback through the LCD display and a Flask web page, while a buzzer alerts the user when one or more conditions are not within the acceptable range. Overall, the system improves efficiency, reduces resource wastage, and reduces manual effort.

2. Introduction

2.1 Background

To improve food security and rely less on imported foods, Singapore has plans for local farms to supply 20 percent of local consumption of fibre and 30 percent of local consumption of protein by 2035, and as of 2024, local consumption of Singapore-produced food items in the fibre category stands at 8 percent and 26 percent in the protein category. Singapore is close to hitting the target of 30 percent of local consumption of protein; however, the local consumption of fibre is still far off the target, and vegetables are one of the main sources of fibre. To do so, the Singapore government has implemented indoor plant cultivation, such as vertical farms and controlled-environment agriculture, as Singapore has limited land space for agriculture, so every plot of land must be used to its fullest extent. Furthermore, these farming methods require good environmental conditions, including temperature, lighting, and water levels for optimal plant growth.

However, in some indoor plant cultivation set-ups, the plants are monitored manually. This approach can lead to inconsistent plant care, inefficient use of resources, and delayed responses to adjust the conditions. With the use of IoT, it is possible to automate environmental monitoring and provide real-time feedback to improve efficiency and sustainability.

2.2 Problem Statement

Indoor plant cultivation requires stable environment conditions such as suitable temperature, lighting, and water level. However, in many indoor farming setups, farmers monitor the plant environment manually which can lead to delay response and inefficiencies resources usage and reduce plant growth quality. Also, in vertical farms, some plants do not receive enough light for optimal growth as the above plants may be blocking the sunlight.

Our Automated Plant Monitoring and Alert System will be able to solve these issues. We have sensors to detect if the surrounding temperature and light are optimal for plant growth, and to measure if the water level is enough. Additionally, if the light intensity value from the light sensor is too low, the LED brightness will automatically increase, users can also manually adjust the LED brightness using the rotary angle sensor. There will also be a buzzer alerting users to respond when one of more of the conditions are not okay. Our project aims to reduce resource wastage and improve plant growth quality and efficiency by having automated plant monitoring, which furthermore lead to sustainability.

3. Project Objectives

- Environmental monitoring: Design a system that can continuously monitor the surrounding plant temperature, light intensity, and water level using temperature sensor, grove light sensor, and ultrasonic sensor to ensure suitable conditions for the plant to grow.
- Automatic light control: To automatically adjust the LED brightness base on the light intensity measured by the grove light sensor, while still allowing manual override adjustment when the user is using the rotary angle sensor to adjust the LED brightness.
- Real-time user feedback: To display current system status which include temperature, light intensity, water level, using 16x2 LCD display.
- Alert system: When the conditions are not suitable for the plant grow, the buzzer will be on, prompting the user to intervene.
- Data storage: All the data will be stored in MariaDB database for future reference and analysis.
- Web monitoring: Allow users to see the data that is recorded, and to be used for further analysis.
- Resource efficiency: To show how IoT can help to improve indoor farming efficiency by reduce the manual checking.

4. Proposed Solution

4.1 Solution Overview

This project is an automated system to help monitor and take care of indoor plant cultivation efficiently. This system helps indoor plant farmers to know whether the plant conditions are suitable for plant growth without constantly checking.

This system uses multiple sensors to check different conditions around the plants. A temperature sensor is used to measure the surrounding plant temperature to ensure it is within a suitable range for plant growth. This is very important, as different plants have different temperature ranges to grow well. In this project, the acceptable temperature range is set between 20°C and 35°C.

Additionally, we have a Grove light sensor to measure the surrounding light intensity, making sure the plants are receiving sufficient light. The acceptable effective light range is set at more than 300; effective light is the light intensity value from the light sensor and the LED brightness added up together.

Furthermore, we have an ultrasonic sensor to measure the distance between the sensor and the water surface to calculate the amount of water left in the tank, ensuring there is enough water. Since ultrasonic sensors cannot detect liquid, we will place a very thin plastic sheet on the water surface. This allows the ultrasonic sensor to measure water levels more accurately. The acceptable distance range between the sensor and the water surface is less than 20 centimetres.

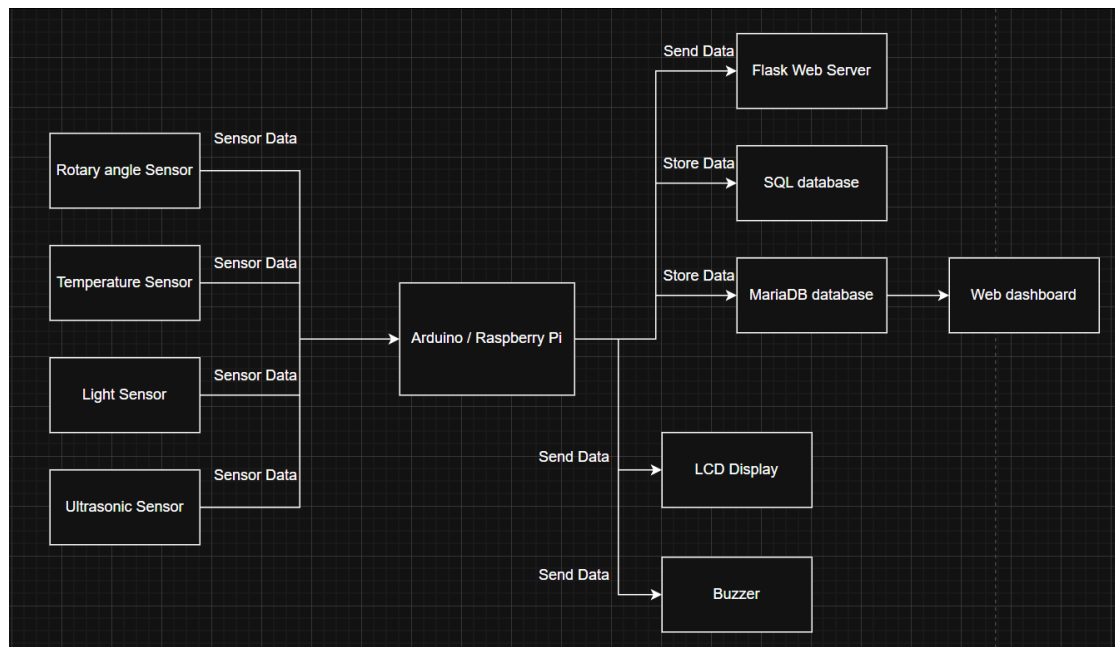
Lastly, we use a rotary angle sensor, which allows users to manually rotate it to adjust the brightness of the LED.

The system can automatically adjust the LED brightness based on the light sensor readings, so users do not need to manually adjust the LED when needed. When the light sensor detects that the light intensity is too low, the LED brightness increases automatically, and when there is sufficient light, the LED brightness is reduced to save energy. In addition, there is an override feature; when the rotary angle sensor is adjusted manually, it takes control of the LED brightness instead of the automatic adjustment.

An Arduino microcontroller is used to collect data from all the sensors and control the LCD display and buzzer. The LCD display shows the plant conditions, such as temperature, light, water level, and whether the conditions are suitable for plant growth. If the conditions are not suitable, the buzzer will alert the user, prompting them to adjust the temperature, light, or water level.

The sensor data is then sent to the Raspberry Pi using serial communication. The Raspberry Pi stores the data in a MariaDB database. The data sent to the Raspberry Pi is uploaded to the Flask web interface, which displays the recorded data in a neat table format for further analysis.

4.2 System Architecture



This diagram above shows how the data is being collected and being used.

Firstly, all the sensors are placed. These sensors are ultrasonic, temperature, light, and rotary angle sensors. Each sensor will collect data and information from the surroundings. Then all the data will be sent to the Arduino. The Arduino will read the information and decide what action to take, such as displaying the information, making the buzzer sound if the environmental conditions are not suitable for plant growth, and increasing the brightness of the LED. After that

the Arduino will send the data to the Raspberry Pi using serial communication. A Python code will be run to let Raspberry Pi process the data that is sent by the Arduino and store the data into the MariaDB database.

Lastly, another Python code will be run so that the data will be uploaded from the database to display on the Flask web interface in a neat table to view and for further analysis.

5. Methodology

5.1 Hardware Setup

Hardware List:

Component	Quantity	Purpose
Arduino Uno	1	Controller for sensor reading
Grove Temperature Sensor	1	Temperature sensor to measure temperature
Grove Ultrasonic Sensor	1	Ultrasonic sensor to measure water levels
Raspberry Pi	1	Processing and database handling
Grove light sensor	1	Measure the light intensity
Grove rotary angle sensor	1	To allow user to adjust the brightness of LED using the knob
LED	1	To simulate the LED grow light
Buzzer	1	To alert the user when the environment conditions are not suitable
16x2 LCD display	1	Display system status and data readings

Component	Quantity	Purpose
Jumper wire	as much as needed	To connect the sensor to the Arduino

For this project, all the sensors and output devices from the above table are connected to the Arduino Uno to collect data from the surrounding environment.

The Grove temperature sensor is connected to the analog pin A0 to measure the surrounding temperature of the plant. The Grove light sensor is connected to analog pin A1 to measure the ambient light intensity. The Grove rotary angle sensor is connected to analog pin A2 to allow the user to adjust the LED brightness. The Grove ultrasonic sensor is connected to digital pin D7. It is positioned above the water tank to measure the distance between the sensor and the water level surface. An LED is connected to digital pin D3 (PWM) to simulate the plant grow light. A buzzer is connected to digital pin D4 to provide an alert when the environmental conditions are not suitable for plant growth. A 16X2 LCD display is connected using the I2C interface pins A4 (SDA) and A5 (SCL) respectively. The LCD display shows real-time information about the environmental conditions.

5.2 Software Setup

Software Requirements:

- The Arduino IDE is use for writing, compile, and uploading the program to the Arduino uno using C++ programming language.
- Python 3 for scripting on Raspberry Pi, which is used to receive and process information from Arduino.
- The pyserial library is used in python 3 to establish serial communication between Arduino and Raspberry Pi.
- The pymysql library is used in python 3 to connect to the MariaDB database.
- The flask framework is used to create a web interface that retrieves data from the database and displays it on a web page.
- The MariaDB database is use as a data storage platform.
PhpMyAdmin to manage MariaDB database.

6. Design and Implementation

6.1 System Design

The system is designed based on an IoT architecture where data is collected, processed, transmitted, stored, and displayed.

Multiple sensors are connected to an Arduino microcontroller to monitor the plant environment. A Grove light sensor measures the surrounding light intensity, a Grove temperature sensor measures temperature, an ultrasonic sensor measures the distance between the top of the water tank where the sensor is located and the water level surface, and a rotary angle sensor is used for adjusting the LED brightness. A PWM LED is used to function as the plant grow light, and a buzzer is used to sound and alert users when one or more conditions are not within the acceptable range.

The Arduino then reads the sensor values and performs the basic processing, such as converting temperature to degrees Celsius using thermistor mathematics, mapping the LED brightness using the PWM duty cycle, and determining whether the temperature, light, and water level values fall within their acceptable ranges. Based on the conditions, the Arduino will decide whether to increase or decrease the LED brightness, follow the user setting from the rotary angle sensor, or activate the buzzer.

The processed data will be sent to the Raspberry Pi through serial communication via the `/dev/ttyACM0` port.

On the Raspberry Pi, a Python program reads the data sent by the Arduino, extracts the temperature, light level, water level, LED brightness, and alert status, and stores this information in the MariaDB database.

A Flask web page is used, and the Raspberry Pi retrieves the stored data from the database and displays it in a table format. This allows users to monitor real-time data more easily through a web browser and use the data for further analysis.

6.2 Sensor Integration

All the sensors and components are connected to the 5V and GND pins on the Arduino through a breadboard.

The Grove temperature sensor is connected to analogue pin A0; it is an analogue input sensor that measures the surrounding temperature.

The Grove light sensor is connected to analogue pin A1. It is an analogue input sensor that measures surrounding light intensity.

The ultrasonic sensor is connected to digital pin D7. It is a digital input sensor that measures the distance from the top of the tank where the sensor is located at to the water level surface.

The rotary angle sensor is connected to analogue pin A2. It is an analogue input sensor that allows the user to set preferred light from 0 to 1023.

The buzzer is connected to digital pin D4. It is a digital output sensor that will sound if the condition is not suitable for the plant to grow.

The LED is connected to digital pin D3, which supports PWM output. It can act as an analogue output, which allows the Arduino to control the brightness of the LED.

The 16X2 LCD display is connected to I2C interface pins A4 (SDA) and A5 (SCL) respectively. The LCD display shows real-time data and data status.

The Arduino then reads the values from the temperature, light, rotary angle, and ultrasonic sensors. These raw values are calculated and processed using thermistor mathematics, PWM mapping for the LED duty cycle, and distance calculation in centimetres based on the echo time of the ultrasonic sensor. The processed data is then displayed on the serial monitor and the LCD display.

6.3 Data Processing

The Arduino performs data processing after reading the raw values from the sensor. Each sensor is converted, evaluated, and compared to the predefined suitable condition to ensure the environment is suitable for the plant growth.

For the temperature measurement, the analog value from the thermistor is converted into degrees Celsius using the thermistor formula. The acceptable range of the temperature is from 20 to 35 degrees Celsius. If the temperature is lower than 20 degrees or higher than 35 degrees, the condition is not suitable, and the buzzer will be turned on.

For the light monitoring, the Arduino reads the ambient light value from the light sensor. The LED light value is also considered, as it provides additional light for the plant. The system calculates the total light using this formula: $\text{effective light} = \text{ambient light} + \text{LED light}$. If the effective light is greater than 300, the light condition is suitable for the plant. If it is less than 300, the system will automatically increase the LED brightness. When there is enough ambient light, the system will automatically decrease the LED brightness to save energy.

The rotary angle sensor allows the user to manually control the LED brightness. When the user turns the knob, the manual adjustment will override the automatic light control.

For the water level, the ultrasonic sensor measures the distance between the sensor and the plastic sheet on the water surface. If the distance is more than 20 cm, the water level is considered low, and the buzzer will ring. If the distance is less than 20 cm, the water level is considered enough.

After all the values are checked and processed, the system will decide whether the overall condition is good. If any of the conditions are not within their acceptable range, the buzzer will turn on, the LCD will display an "N" beside the value, and an alert will be sent through the serial communication to the Raspberry Pi. If all conditions are good, the buzzer will turn off and the LCD will display a "Y" beside the value, and an alert will be sent through the serial communication to the Raspberry Pi.

Overall, the data from the sensor is processed by the Arduino, and then it is transmitted to the Raspberry Pi through serial communication for data storage and web display.

6.4 Database Implementation

The project uses MariaDB as the database management system. It runs on the Raspberry Pi and stores sensor readings collected from the Arduino.

The sensor reading data received from the Arduino through serial communication is then processed using a Python program running on the Raspberry Pi. The data insertion into the database is done using Python's PyMySQL library. The data from the database can be retrieved and displayed on a web interface using Flask.

A table with nine columns is created in the database to store the sensor readings. The nine columns are:

- SNo. – auto-increment serial number for each data record
- NYPID – our admin numbers
- Name – our names
- Date – timestamp of data collection
- Temperature – temperature value in degrees Celsius
- Light – light sensor value
- Water – distance between the ultrasonic sensor and the water surface
- LED – brightness
- Alert – alert message

This database allows all sensor readings to be stored and later displayed through the Flask web interface on a web page, which allows users to monitor, research, and analyse the data. Also, each record is inserted automatically with a timestamp for tracking the system history.

6.5 Web Interface

A Python program is run on the Raspberry Pi to start the Flask application, and the user will be redirected to a browser where the Flask web interface can be accessed.

The Flask framework uses Jinja2 templates to render HTML pages. The data is retrieved from the database and passed into the templates, which generate the Flask web interface.

The web interface displays the data in a table format, showing values such as temperature, light, water level, LED brightness, and alert messages. The web page updates automatically when new data is inserted into the database.

7. Testing and Results

7.1 Test Cases

Test Case	Input / Condition	Expected Output	Actual Output	Pass/Fail
TC1	Temp >20 and <35 and Light > 300 and Water level (distance) <20	"All conditions OK -> Buzzer OFF" message	"All conditions OK -> Buzzer OFF" message	Pass
TC2	Temp <20 or >35 OR Light < 300 OR Water level (distance) >20	"Conditions NOT OK -> Buzzer ON" message	"Conditions NOT OK -> Buzzer ON" message	Pass
TC3	Effective light $\leq$ 300 (dark environment)	LED brightness increases automatically	LED brightness increase and light condition become ok	Pass
TC4	Effective light $\geq$ 300 (bright environment)	LED brightness decreases automatically	LED brightness decrease and light condition become ok	Pass
TC5	Water distance $\geq$ 20 (low water level)	"Condition not ok ->buzzer on" message	Alert message displayed and buzzer activated	Pass
TC6	User rotates rotary angle sensor	Manual LED brightness overrides automatic control	LED Brightness changed according to rotary input even the condition is not okay	Pass
TC7	Users use rotary angle sensor	The LED will be brighter or dimmer when the user turns the knob	LED brightness changed based on user preference	Pass

7.2 Challenges Encountered and Resolution

During the project, we encountered many problems at different stages of development.

The first challenge was understanding how to connect and use the sensors. At the beginning, my teammate and I were very unfamiliar with the light and rotary angle sensors. We did not know their functions or how to integrate them into one project. To overcome this, we referred to the official Grove website to understand how to connect and use them. Through multiple tests, we successfully integrated them into the project.

Another challenge was programming the Arduino. As we were not very familiar with the C++ programming language, it was difficult to write the code. We learned the required concepts through the notes and with the help of the teacher. After several attempts and testing, the program finally worked.

When connecting the Arduino to the Raspberry Pi, we faced an issue where the serial output could not be displayed. Even though the code showed no syntax errors, the data was not displayed. We asked our teacher, who suggested that it might be a logic issue in the code. After modifying the code and with guidance from the teacher, we managed to get the serial output working properly.

We also encountered many string and formatting errors in the Python program. We debugged the code step by step, and eventually it worked.

Finally, there were errors when we were trying to run the Flask web application. At first, we checked the code logic and indentation, and everything seemed correct, but the problem still existed. After asking the teacher, we discovered that the file was saved in the wrong location and not in the template folder. Once we corrected this, the web application worked.

Through these challenges, we learned the importance of teamwork, perseverance, discipline, and problem-solving skills. These experiences helped us understand the system better.

8. Conclusion

Overall, this project successfully developed an IoT-based automated smart plant monitoring and control system. This system can monitor temperature, light intensity, and water level in real time and automatically adjust the LED grow light based on the surrounding light intensity. When one condition falls outside the acceptable range, such as a temperature below 20°C or above 35°C, the buzzer will be activated to alert the user.

The system also allows manual control using a rotary angle sensor, which overrides the automatic LED brightness when needed. All sensor data is transmitted from the Arduino to the Raspberry Pi, stored in a MariaDB database, and displayed on a Flask web page for the user to view real-time data.

During the project, we faced challenges such as sensor integration, coding problems, and Flask web issues. Through troubleshooting, research, and asking the teacher for help, these problems were solved. This project helped us understand IoT systems and programming better. We also learned the importance of teamwork, perseverance, discipline, and problem-solving skills.

For future improvements, the system can automatically help the user add water into the water tank instead of only providing an alert and automatically increase or decrease the temperature instead of only giving a warning.

Overall, this project demonstrates how IoT technology can be used to increase the efficiency of indoor farming and reduce manual effort, which is important for sustainability and food security in Singapore.

9. References

- Mohan, M. (2026, January 20). Singapore replaces “30 by 30” food sustainability goal with targets for specific food categories. *CNA*.
<https://www.channelnewsasia.com/singapore/30-30-food-sustainability-goal-replace-fibre-protein-5441756>
- *Grove - Light Sensor* | *Seed Studio Wiki*. (2023, January 5).
https://wiki.seeedstudio.com/Grove-Light_Sensor/
- *Grove - Rotary Angle Sensor* | *seed Studio Wiki*. (2023, January 19).
https://wiki.seeedstudio.com/Grove-Rotary_Angle_Sensor/

10. Appendix

Appendix A – Arduino code

```

#include <math.h>
#include "rgb_lcd.h"
#include "Ultrasonic.h"

Ultrasonic ultrasonic(7);

const int B = 4275;
const int R0 = 100000;
const int TempSensor = A0;

const int lightPin = A1;
const int rotaryPin = A2;
const int ledPin = 3;
const int buzzerPin = 4;

void setup() {
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT);
  pinMode(buzzerPin, OUTPUT);
  lcd.begin(16, 2);
}

void loop() {
  //thermistor math
  int a = analogRead(TempSensor);
  float R = (1023.0 / a - 1.0) * R0;
  float temperature = 1.0 / (log(R / R0) / B + 1.0 / 298.15) - 273.15;

  //light sensor
  int lightValue = analogRead(lightPin);

  //rotary angle sensor
  int rotaryValue = analogRead(rotaryPin);

  //automatic led brightness
  int autoBrightness = map(lightValue, 0, 1023, 255, 0);

  int brightness;
  //override using rotary angle sensor
  if (rotaryValue > 50) {
    // If knob is turned, override automatic control
    brightness = map(rotaryValue, 0, 1023, 0, 255);
  } else {
    //otherwise use automatic brightness
    brightness = autoBrightness;
  }
}

```

```

analogWrite(ledPin, brightness);

//water level
long distanceCM = ultrasonic.MeasureInCentimeters();

int effectiveLight = lightValue + brightness;

//condition check
bool tempOK = (temperature >= 20 && temperature <= 35);
bool lightOK = (effectiveLight > 300);
bool waterOK = (distanceCM < 20);

//lcd output
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("T:");
lcd.print(temperature, 1);
lcd.print("C ");
lcd.print(tempOK ? "Y" : "N");

lcd.print(" L:");
lcd.print(lightValue);

lcd.print(lightOK ? "Y" : "N");

lcd.setCursor(0, 1);
lcd.print("W:");
lcd.print(distanceCM);
lcd.print("cm ");
lcd.print(waterOK ? "Y" : "N");

lcd.print(" LED:");
lcd.print(brightness);

//serial output
Serial.print("TEMP:");
Serial.print(temperature, 1);
Serial.print("C ");
Serial.println(tempOK ? "Y" : "N");

Serial.print("LIGHT:");
Serial.print(lightValue );
Serial.println(lightOK ? "Y" : "N");

Serial.print(" WATER:");
Serial.print(distanceCM);
Serial.print("cm ");
Serial.println(waterOK ? "Y" : "N");

Serial.print("LED:");
Serial.println(brightness);

```

```

//buzzer alert
if (!tempOK || !lightOK || !waterOK) {
  digitalWrite(buzzerPin, HIGH);
  lcd.setRGB(255, 0, 0);
  Serial.println("ALERT: Condition NOT OK -> Buzzer ON");
} else {
  digitalWrite(buzzerPin, LOW);
  lcd.setRGB(0, 255, 0);
  Serial.println("ALERT: All conditions OK -> Buzzer OFF");
}

delay(1000);
}

```

Appendix B – Python Code (Raspberry Pi)

```
#import serial library
import serial
#read serial data coming from Arduino
ser = serial.Serial('/dev/ttyACM0', 9600)

while True:
    line = ser.readline()
    line = line.decode('utf-8', errors='ignore').strip()

    if line == "":
        continue

    if line.startswith("TEMP:"):
        x = line.replace("TEMP:", "")
        x = x.replace("C", "")
        parts = x.split()
        temp_value = parts[0]
        temp_status = parts[1]
        print("Temperature =", temp_value, "C", temp_status)

    elif line.startswith("LIGHT:"):
        x = line.replace("LIGHT:", "")
        x = x.replace("", "")
        light_value = x[:-1]
        light_status = x[-1]
        print("Light Level =", light_value, light_status)

    elif line.startswith("WATER:"):

        x = line.replace("WATER:", "")
        x = x.replace("cm", "")
        parts = x.split()
        water_value = parts[0]
        water_status = parts[1]
        print("Water Level =", water_value, "cm", water_status)

    elif line.startswith("LED:", ""):
        led_pwm = line.replace("LED:", "")
        print("LED PWM =", led_pwm)

    elif line.startswith("ALERT:"):
        alert_msg = line.replace("ALERT:", "")
        print("ALERT =", alert_msg)
        print("-----")
```

Appendix C- flask code

```

from flask import Flask, render_template
app = Flask(__name__, template_folder='templates')

import pymysql
#connecting to a MySQLdb
db=pymysql.connect(user='Nypuser', passwd='password', host='localhost',
db='ProjPart2')
cursor = db.cursor()

cursor.execute("Select * from ProjPart2.Part2Data")
results = cursor.fetchall()

@app.route('/<LiHengZiqian>')
def index(LiHengZiqian):
    message = LiHengZiqian + "'s Project Part 2 Data"
    return render_template('sql-data.html2', message=message, results=results)

if __name__ == '__main__':
    app.run()

<!DOCTYPE html>

<html>
<head>
    <title>Flask Jinja Templates for Databases </title>
</head>

<body>

<h1> EG1P54 Sensor Data </h1>
<h2> {{ message }} </h2>

<table >
    <tr >
        <th>No.</th>
        <th>NYP ID</th>
        <th>Name</th>
        <th>Time Stamp</th>
        <th>Temperature</th>
        <th>Light Level</th>
        <th>Water Level</th>
        <th>LED</th>
        <th>Alert</th>

    </tr>
    {% for row in results %}
    <tr>
        <td>{{ row[0] }}</td>
        <td>{{ row[1] }}</td>
        <td>{{ row[2] }}</td>
        <td>{{ row[3] }}</td>
        <td>{{ row[4] }}</td>
        <td>{{ row[5] }}</td>
        <td>{{ row[6] }}</td>
        <td>{{ row[7] }}</td>
        <td>{{ row[8] }}</td>
    </tr>
    {% endfor %}
</table>

</body>
</html>

```

Appendix D- Raspberry Pi to database

```
#serial comm library
import serial
#pymysql library for Python
import pymysql
#read serial data coming from Arduino
ser = serial.Serial('/dev/ttyACM0', 9600)
#connecting to a MySQLdb
db=pymysql.connect(user='Nypuser', passwd='password', host='localhost', db='ProjPart2')
cursor = db.cursor()

temp_value = None
light_value = None
water_value = None
led_pwm = None
alert_msg = None

try:
    while True:
        line = ser.readline().decode('utf-8', errors='ignore').strip()
        if line== "":
            continue

        if line.startswith("TEMP:"):

            if line.startswith("TEMP:"):
                x = line.replace("TEMP:", "").replace("C", "")
                parts = x.split()
                if len(parts) >= 2:
                    temp_value = parts[0]
                    temp_status = parts[1]
                    print("Temperature =", temp_value, "C", temp_status)

            elif line.startswith("LIGHT:"):
                x = line.replace("LIGHT:", "").replace(" ", "")
                if len(x) >= 2:
                    light_value = x[:-1]
                    light_status = x[-1]
                    print("Light Level =", light_value, light_status)

            elif line.startswith("WATER:"):
                x = line.replace("WATER:", "").replace("cm", "")
                parts = x.split()
                if len(parts) >= 2:
                    water_value = parts[0]
                    water_status = parts[1]
                    print("Water Level =", water_value, "cm", water_status)

            elif line.startswith("LED:"):
                led_pwm = line.replace("LED:", "")
                print("LED PWM =", led_pwm)

            elif line.startswith("ALERT:"):
                alert_msg = line.replace("ALERT:", "")
                print("ALERT =", alert_msg)
                print("-----")

        #only insert if we got everything
        if None not in (temp_value, light_value, water_value, led_pwm, alert_msg):
            cursor.execute(
                "INSERT INTO Part2Data (Temp, Light, Water, LED, Alert) "
                "VALUES (%s, %s, %s, %s, %s)",
                (temp_value, light_value, water_value, led_pwm, alert_msg)
            )
            db.commit()
            print("Database updated successfully\n")
        else:
            print("Skipping insert, missing data")

except Exception as e:
    print("Error: ", e)
    db.rollback()
db.close()
```